

Я создала UI в qt 4.8.6 с++ и добавила туда кодом toolbar, чтобы красивенько у меня иконки справа располагались. А теперь хочу вынести в отдельный файл Settings, собственно, который будет внутри содержать класс Settings, где будут храниться все настройки с кнопками и прочее. И туда хочу перенести создание тулбара. И я пыталась через New Settings это делать и через unique_ptr. Один фиг через новый класс не создается ничего

Thought for 1 second

User is creating a Qt 4.8.6 UI with C++ and wants to move toolbar creation into a separate Settings class. They tried New Settings and unique_ptr but the toolbar isn't being created.

Let me help them figure out the issue. The common problems are:

1. Parent-child relationship - if Settings isn't a QObject or doesn't have proper parent, the toolbar won't show

2. Ownership issues with unique_ptr

3. Not calling the setup method

4. Toolbar needs to be added to the main window

Let me provide a proper example.

Проблема почти всегда в одном из трёх мест. Давай разберём по порядку.

Почему «не создаётся»

QToolBar (как и любой виджет) не появится сам по себе, пока его не добавили в родителя через addToolBar() / addWidget(). Если ты делаешь Settings обычным классом (не QObject / QWidget) и создаёшь QToolBar внутри — он создастся в памяти, но на экране его не будет, пока ты не передашь его главному окну.

Plus unique_ptr<Settings> не подойдёт, если Settings — QObject : у QObject удаление через unique_ptr конфликтует с родительской иерархией (двойное удаление). Для QObject -наследников используй обычный указатель с parent или QScopedPointer / QSharedPointer .

Правильный вариант №1 — Settings как QObject с методом setup

settings.h

```
cpp

#ifndef SETTINGS_H
#define SETTINGS_H

#include <QObject>
#include <QToolBar>
#include <QMainWindow>

class Settings : public QObject
{
    Q_OBJECT
public:
    explicit Settings(QMainWindow *parent);
    QToolBar* toolbar() const { return m_toolbar; }

private:
    void createToolBar();

    QMainWindow *m_mainWindow;
    QToolBar *m_toolbar;
};
```

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our [cookie policy](#). You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies

```
#endif // SETTINGS_H

settings.cpp

cpp

#include "settings.h"
#include <QAction>
#include <QIcon>

Settings::Settings(QMainWindow *parent)
    : QObject(parent)
    , m_mainWindow(parent)
    , m_toolbar(0)
{
    createToolBar();
}

void Settings::createToolBar()
{
    m_toolbar = new QToolBar(tr("Settings"), m_mainWindow); // parent!
    m_toolbar->setMovable(false);

    // Иконки прижимаем вправо: добавляем "распорку" слева
    QWidget *spacer = new QWidget(m_toolbar);
    spacer->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Preferred);
    m_toolbar->addWidget(spacer);

    QAction *actSettings = m_toolbar->addAction(QIcon(":/icons/settings.png"), tr("Settings"));
    QAction *actHelp      = m_toolbar->addAction(QIcon(":/icons/help.png"),      tr("Help"));

    connect(actSettings, SIGNAL(triggered()), this, SLOT(/*...*/));
    connect(actHelp,     SIGNAL(triggered()), this, SLOT(/*...*/));

    // ВАЖНО: добавить тулбар в главное окно
    m_mainWindow->addToolBar(Qt::TopToolBarArea, m_toolbar);
}
```

В главном окне:

```
cpp

// mainwindow.cpp
#include "settings.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
{
    ui->setupUi(this);

    m_settings = new Settings(this); // parent = this, удалится автоматически
}
```

Почему у тебя не работало

Что ты сделала	Почему не работает
std::unique_ptr<Settings> s = new Settings(...) без parent	ToolBar создаётся, но addToolBar не вызывается — на экране пусто

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our [cookie policy](#). You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies

Что ты сделала	Почему не работает
Settings — обычный класс, не QObject, и внутри new QToolBar без parent	Утечка/окно само не покажет тулбар
unique_ptr<Settings> где Settings : QObject с parent	Двойное удаление при выходе из scope (Qt удалит как ребёнка + unique_ptr удалит)
Забыла #include "settings.h" и просто объявила	Линкер не свяжет, но код компилируется — «ничего не создаётся»

Ключевые правила

- 1. Любой QObject/QWidget должен иметь parent, иначе сам не удалится и не покажется.
- 2. QMainWindow::addToolBar() — обязательный вызов, иначе тулбар не появится.
- 3. Не мешай unique_ptr и QObject -иерархию — либо то, либо другое.
- 4. Если класс Settings создаёт виджеты, пусть он будет QObject (или QWidget), тогда connect и parent работают «из коробки».

Если скинешь свой реальный код Settings и вызов из главного окна — покажу точное место, где ломается.

а еще методы есть? а то я буду пробовать на компе без интернета, чтобы точно получилось если что

Thought for 1 second User wants alternative methods that will definitely work offline. Let me give several approaches. Да, давай дам тебе несколько рабочих вариантов — попробуешь все, какой-нибудь точно зайдёт. Всё офлайн, ничего скачивать не надо.

Вариант 1. Settings как QObject (самый правильный, повторю кратко)

```
cpp
class Settings : public QObject {
    Q_OBJECT
public:
    explicit Settings(QMainWindow *mw);
};
```

В конструкторе: m_toolbar = new QToolBar(this) не сработает — toolbar нельзя вешать на QObject. Родитель тулбара — обязательно QWidget . Поэтому:

```
cpp
m_toolbar = new QToolBar(m_mainWindow);
...
m_mainWindow->addToolBar(m_toolbar);
```

Вариант 2. Settings как QWidget (тулбар как «шапка» виджета)

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our cookie policy. You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies

Если не хочешь трогать addToolBar , сделай Settings виджетом и вставь его в centralWidget через layout. Иконки будут справа — просто добавь QSpacerItem слева.

```
cpp

class Settings : public QWidget {
    Q_OBJECT
public:
    explicit Settings(QWidget *parent = 0);
};

cpp

Settings::Settings(QWidget *parent) : QWidget(parent) {
    QHBoxLayout *lay = new QHBoxLayout(this);
    lay->setContentsMargins(0,0,0,0);

    lay->addStretch(1); // прижимает всё вправо

    QPushButton *btn1 = new QPushButton(this);
    btn1->setIcon(QIcon(":/icons/settings.png"));
    btn1->setAutoRaise(true);
    lay->addWidget(btn1);

    QPushButton *btn2 = new QPushButton(this);
    btn2->setIcon(QIcon(":/icons/help.png"));
    btn2->setAutoRaise(true);
    lay->addWidget(btn2);
}
```

В главном окне:

```
cpp

m_settings = new Settings(this);
ui->centralWidget->layout()->addWidget(m_settings); // или свой layout
```

Вариант 3. Settings как обычный класс-хелпер без QObject

Если не хочешь возиться с тос и Q_OBJECT — сделай класс без сигналов/слотов, только фабрику.

settings.h

```
cpp

#ifndef SETTINGS_H
#define SETTINGS_H

#include <QMainWindow>

class Settings {
public:
    static void install(QMainWindow *mw); // создаёт и вешает тулбар
};

#endif
```

settings.cpp

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our [cookie policy](#). You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies

```
cpp

#include "settings.h"
#include <QToolBar>
#include <QAction>
#include <QIcon>
#include <QWidget>
#include <QSizePolicy>

void Settings::install(QMainWindow *mw) {
    QToolBar *tb = new QToolBar("Settings", mw);
    tb->setMovable(false);

    QWidget *spacer = new QWidget(tb);
    spacer->setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Preferred);
    tb->addWidget(spacer);

    tb->addAction(QIcon(":/icons/settings.png"), "Settings");
    tb->addAction(QIcon(":/icons/help.png"), "Help");

    mw->addToolBar(Qt::TopToolBarArea, tb);
}
```

Вызов в MainWindow:

```
cpp

#include "settings.h"
...
ui->setUpUi(this);
Settings::install(this);
```

Минус: сигналы придётся цеплять снаружи, но зато не нужен **moc** — просто добавь settings.cpp в .pro . Очень надёжно для «попробовать офлайн».

Вариант 4. Отдельный .ui для Settings

Можно вынести тулбар в свой settings.ui :

- 1. В Qt Designer создай новый QWidget (не MainWindow).
- 2. Накидай туда QHBoxLayout → Horizontal Spacer → кнопки.
- 3. Сохрани как settings.ui .
- 4. uic (или qmake) сгенерирует ui_settings.h .

Затем:

```
cpp

#include "ui_settings.h"

class Settings : public QWidget {
    Q_OBJECT
public:
    explicit Settings(QWidget *parent = 0) : QWidget(parent), ui(new Ui::Settings) {
        ui->setUpUi(this);
    }
    ~Settings() { delete ui; }
private:
```

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our [cookie policy](#). You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies

```
Ui::Settings *ui;
};
```

И в главном окне — как в варианте 2.

Не забудь в .pro добавить:

```
text

FORMS += settings.ui
HEADERS += settings.h
SOURCES += settings.cpp
```

Частые грабли (проверь у себя!)

- 1. addToolBar не вызывается → тулбар создан, но невидим.
- 2. Родитель new QToolBar() не передан → тулбар существует, но не привязан к окну.
- 3. Q_OBJECT есть, а moc не отработал → undefined reference to vtable. Лечится: qmake заново, проверить, что .h в HEADERS в .pro.
- 4. Класс Settings объявлен в .cpp главного окна → moc не видит. Выноси header отдельно.
- 5. unique_ptr<Settings>, где Settings : QObject и parent = this → двойное удаление, краш при закрытии. Просто Settings *m_settings; c parent.
- 6. Забыла вернуть ui->setUpUi(this) до Settings::install(this) → centralWidget ещё null.
- 7. Иконки не грузятся → путь к ресурсам: ./icons/... работает только если в .pro есть RESOURCES += res.qrc.

Что реально «точно работает» офлайн

Самый бесппроблемный путь, если хочешь просто чтобы заработало:

- 1. Возьми Вариант 3 (обычный класс + статический метод install).
- 2. В .pro добавь только SOURCES += settings.cpp и HEADERS += settings.h.
- 3. Никаких moc, unique_ptr, Q_OBJECT — нечему ломаться.
- 4. Тулбар появится сразу после ui->setUpUi(this).

Когда уже увидишь тулбар на экране — перейдешь к Варианту 1, чтобы повесить сигналы на кнопки.

Cookie Settings

We use cookies to provide and improve services and ensure security. Click to view our [cookie policy](#). You can choose to accept all or only necessary cookies, which may affect some functionalities.

Necessary cookies only

Accept all cookies